```
→  pact-js-example git:(update-packages) ✗ npm run test:graphql:consumer

> pact@1.0.0 test:graphql:consumer
> jest graphql/consumer/graphql-client-contract.spec.js


 RUNS  graphql/consumer/graphql-client-contract.spec.js
 RUNS  graphql/consumer/graphql-client-contract.spec.js
2023-12-22T20:01:53.245733Z  INFO tokio-runtime-worker pact_mock_server::hyper_server: Received request PO
ST /graphql
2023-12-22T20:01:53.246551Z  INFO tokio-runtime-worker pact_matching: comparing to expected HTTP Request (
 method: POST, path: /graphql, query: None, headers: Some({"Content-Type": ["application/json"]}), body: P
resent(206 bytes, application/json) )
2023-12-22T20:01:53.251380Z  INFO tokio-runtime-worker pact_mock_server::hyper_server: Request matched, se
nding response
 PASS  graphql/consumer/graphql-client-contract.spec.jsels::pact: Note: Existing pact is an older specific
  GraphQL example
    When a query to list all movies on /graphql is made
      ✓ returns the correct response (24 ms)

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   0 total
Time:        0.658 s, estimated 1 s
Ran all test suites matching /graphql\/consumer\/graphql-client-contract.spec.js/i.
```

Figure 7.5 Test output of running the GraphQL consumer contract test

When the test run completes, Pact will also generate the contract, which is saved in the `pacts` directory. If you also generated the web consumer contract from chapter 4, you should now see a similar folder structure output like this:

```
> consumer-mobile
> consumer-web
> data
> graphql
> pacts
  { } GraphQLConsumer-GraphQLProvider.json
  { } WebConsumer-MoviesAPI.json
> provider
```